

From Bag-of-Words to Fine-Tuned BERT: Modeling fMRI BOLD Responses to Natural Language

1 Introduction

There is a need to understand how the human brain interprets language. Functional Magnetic Resonance Imaging (fMRI) allows researchers to observe how the brain responds to language stimuli by measuring blood-oxygen-level-dependent (BOLD) signals across different brain regions. This project builds on prior work by Jain and Huth (2018), who demonstrated that introducing context into word representations significantly improves the ability to model fMRI responses.

The central question is how different word-embedding strategies affect the prediction of whole-brain BOLD responses during natural story listening, and this project answers it by working through embeddings of increasing sophistication:

1. **Static, one-word-one-vector embeddings:** Bag-of-Words, Word2Vec, and GloVe, which assign every word a fixed vector regardless of context.
2. **A custom contextual encoder:** a small BERT-style transformer trained from scratch via masked language modeling (MLM) directly on the story text, to test whether context-awareness alone (without large-scale pretraining) helps.
3. **Large-scale pretrained and fine-tuned contextual embeddings:** the pretrained `bert-base-uncased` model, and a LoRA fine-tuned variant of it, to test whether contextual representations only pay off once they carry knowledge from a much larger corpus.

For every embedding method, we construct time-aligned feature matrices, downsample them to match fMRI scan rates, and fit a ridge regression model to predict voxel-wise responses, using lagged features to capture the temporal dynamics of neural processing. We compare methods using correlation-based evaluation metrics across two subjects, and finish by applying SHAP and LIME to the best-performing model to ask which specific words drive the best-predicted voxels.

2 Exploratory Data Analysis

This section describes the structure and characteristics of the data used throughout the project.

There are two types of data: whole-brain blood-oxygen-level-dependent (BOLD) signals measured at various points across a set of podcast stories, scanned by fMRI (treated as the “response”), and the raw text of the stories (treated as the “stimulus”).

The response data was collected from two anonymized subjects, “subject 2” and “subject

3,” who were scanned using fMRI while listening to the same set of 101 stories. Each fMRI scan collected signals from 94,251 voxels for subject 2 and 95,556 voxels for subject 3, which suggests individual differences in brain structure and size. Due to this variability, it is not feasible to train a model on one subject’s data and apply it directly to predict another; each subject is therefore modeled independently. We randomly split the 101 stories into roughly 70% training and 30% test stories, train each model, and evaluate on held-out stories per subject.

The stimulus data includes the text segmented into words, along with the time each word was played. Once word embeddings are generated from the text, these timestamps allow each word to be aligned with the corresponding fMRI scan time point, matching the dimensions of the response and stimulus data.

For each story in the training data, word counts per story ranged from 697 to 3,274, and unique-word counts per story ranged from 278 to 1,052. Across the full training set there were 10,356 unique words. As discussed below, having more than 10,000 unique words makes the raw stimulus matrix X very large, which drives several of the dimensionality-reduction choices made for the static embeddings.

3 Embeddings

3.1 Static embeddings: Bag-of-Words, Word2Vec, and GloVe

3.1.1 Bag-of-Words

We begin with a Bag-of-Words embedding. The training data contains 10,356 unique words (the “word list”). For each word, we create a one-hot vector in $\mathbb{R}^{10356}$, where the position corresponding to the word is set to 1 and all other positions are 0. Concatenating these vectors vertically yields an $\mathbb{R}^{\text{wordcount} \times 10356}$ embedding matrix, with column dimensions shared across all stories so that all training data can be combined for a single ridge regression fit.

This high-dimensional embedding poses computational challenges: even after downsampling, the dimensionality remains too large for efficient ridge regression. We therefore restricted the vocabulary to the 300 most frequent words in the training data, giving each word a vector in $\mathbb{R}^{300}$ and matching the dimensionality of the Word2Vec and GloVe embeddings used below, for a fair comparison. The 300-word corpus includes common function words (“and”, “the”), which carry little semantic meaning on their own; we kept them rather than pick up rarer words, since removing them would make X sparser without a clear benefit. To prevent data leakage, the corpus includes only words observed in the training set.

3.1.2 Downsampling and lagging (shared by all embeddings)

Every embedding in this project goes through the same two post-processing steps before ridge regression.

Downsampling. We use the provided `downsample_word_vectors` function to align the embedding’s row dimension with the number of fMRI scans, via Lanczos interpolation. This is equivalent to aggregating the vectors of words played before and after each fMRI scan into

a single new vector. Formally, let $X_{old} \in \mathbb{R}^{wordcount \times d}$ be the original embedding, $X_{new} \in \mathbb{R}^{\#fMRI \text{ scan} \times d}$ the new embedding, t_{old} and t_{new} the corresponding time-point vectors, and w the Lanczos window (here $w = 3$):

$$X_{new} = L(t_{new}, t_{old}, w) X_{old}, \quad L(t_{new}, t_{old}, w) \in \mathbb{R}^{\#fMRI \text{ scan} \times wordcount}$$

$$L(t_{new}, t_{old}, w)_{i,j} = \begin{cases} 1 & \text{if } \frac{t_{new,i} - t_{old,j}}{2} = 0 \\ \frac{w \sin\left(\pi \frac{t_{new,i} - t_{old,j}}{2}\right) \sin\left(\pi \frac{t_{new,i} - t_{old,j}}{2w}\right)}{\pi^2 \left(\frac{t_{new,i} - t_{old,j}}{2}\right)^2} & \text{if } -w \leq \frac{t_{new,i} - t_{old,j}}{2} \leq w \\ 0 & \text{otherwise} \end{cases}$$

We then drop the first 5 and last 10 rows of X_{new} so that its row count matches the number of fMRI scans for each story.

Lagging. A word’s influence on BOLD signal is not instantaneous; subjects retain information about words heard some time ago. For each column of X_{new} we therefore construct lagged copies at delays $[1, 2, 3, 4]$ using the provided `make_delayed` function (only positive lags are used, since the downsampling step already mixes in some information from times the subject had not yet heard). This multiplies the column dimension by 4; e.g. for Bag-of-Words, GloVe, and Word2Vec (each 300-dimensional per word) the final per-story feature matrix is $\mathbb{R}^{\#fMRI \text{ scan} \times 1200}$.

3.1.3 Word2Vec and GloVe

In addition to Bag-of-Words, we used two pretrained embedding methods, Word2Vec and GloVe, which produce dense, low-dimensional vectors that capture semantic relationships between words rather than simply recording word occurrence.

Word2Vec is a neural embedding method that estimates word vectors by modeling a large text corpus. We use the Skip-gram architecture, which predicts surrounding context words given a target word, maximizing

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T \sum_{\substack{-c \leq j \leq c \\ j \neq 0}} \log p(w_{t+j} | w_t), \quad p(w_O | w_I) = \frac{\exp(v'_O{}^\top v_I)}{\sum_{w=1}^{|V|} \exp(v'_w{}^\top v_I)} \quad (1)$$

where T is the number of words in the corpus, w_t the target word, w_{t+j} a context word within the window, v_I the input vector, and v'_O the output vector. We use the pretrained Word2Vec model trained on the Google News corpus (300-dimensional vectors for roughly 3 million words/phrases), mapping each word in a story to its vector and zero-filling out-of-vocabulary words.

GloVe (Global Vectors for Word Representation) learns word vectors from global word-word co-occurrence statistics, minimizing

$$J = \sum_{i,j=1}^V f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 \quad (2)$$

where X_{ij} is the co-occurrence count of words i and j , $f(\cdot)$ down-weights very frequent co-occurrences, $w_i, \tilde{w}_j$ are the word vectors, and $b_i, \tilde{b}_j$ are bias terms. We use the pretrained `glove.6B.300d` vectors (trained on 6 billion tokens).

Both Word2Vec and GloVe embeddings end up with final dimension $[n \times 1500]$ per story (300 features $\times$ 5 lags, including lag 0 for these two), where n is the number of fMRI measurements.

Pretrained embeddings like these bring several benefits over Bag-of-Words: they transfer semantic knowledge learned from massive corpora (e.g. “happy” and “joyful” end up nearby in vector space, unlike in Bag-of-Words), they are dense and low-dimensional rather than sparse and vocabulary-sized (helping avoid the curse of dimensionality), they perform better with the limited amount of story text available here, and they avoid the cost of training word representations from scratch. Table 1 summarizes the differences.

Feature	Bag-of-Words	Word2Vec	GloVe
Dimensionality	High (10,356), reduced to 300	Low (300)	Low (300)
Sparsity	Very sparse (mostly zeros)	Dense	Dense
Training method	Simple counting	Neural network	Matrix factorization
Context capture	None (word presence only)	Local context	Global co-occurrence
Semantic information	None	High	High

Table 1: Comparison of Bag-of-Words, Word2Vec, and GloVe embeddings.

None of these three methods, however, models the fact that a word’s meaning depends on its surrounding words: “model” means something different in “train a machine learning model” than in “he is a fashion model,” and a one-word-one-vector scheme cannot express that difference. The rest of the project investigates whether *contextual* embeddings do better.

3.2 A custom contextual encoder trained from scratch

To generate features that reflect a word’s contextual meaning, we implemented and trained a small BERT-style transformer encoder ourselves, using masked language modeling (MLM, BERT’s original pretraining objective) on the story text.

3.2.1 Architecture

The encoder gives each token contextual meaning through self-attention. Let $X \in \mathbb{R}^{n \times d}$ be the input matrix (one row per token) and $Q, K, V \in \mathbb{R}^{d \times d}$ the query/key/value projection matrices, where n is the number of tokens and d the embedding dimension:

$$\tilde{Q} = XQ \in \mathbb{R}^{n \times d}, \quad \tilde{K} = XK \in \mathbb{R}^{n \times d}, \quad \tilde{V} = XV \in \mathbb{R}^{n \times d}$$

$$A = \text{softmax} \left(\frac{\tilde{Q}\tilde{K}^T}{\sqrt{d}} \right) \in \mathbb{R}^{n \times n}, \quad O = A\tilde{V} \in \mathbb{R}^{n \times d}$$

A reflects the relationship between every pair of tokens via their inner product; the model aggregates $\tilde{V}$ weighted by A and adds it back to the original vector by residual connection, giving each token contextual meaning. Multi-head attention repeats this with several independent Q, K, V sets to capture different types of relationships simultaneously. Each attention block is followed by a position-wise feed-forward network, and the full encoder stacks several such blocks. We tokenize with the pretrained “bert-base-uncased” tokenizer.

Because story text has no natural sentence boundaries in the raw word/timestamp stream, we defined batches by cutting at “sentence breakpoints”: (i) a gap of more than 0.7 seconds since the previous word, (ii) provided the current sentence already has at least 30 words, or (iii) a hard cutoff once a sentence exceeds 100 words. This produced batches that mostly read as coherent, standalone sentences.

To align the resulting feature matrix with fMRI scan times, token-level features belonging to the same word are averaged, then the same downsampling and lagging procedure from Section 2.2 is applied.

3.2.2 Masked language model training

MLM randomly masks tokens and trains the model to predict them from both left and right context; unlike next-word prediction, this lets the model see the full surrounding context. Following the original BERT recipe, 15% of tokens are selected for possible masking; of those, 80% are replaced with [MASK], 10% are replaced with a random vocabulary token, and 10% are left unchanged. This mix keeps the model from only learning to predict at [MASK] positions (which never appear at inference time) and makes it more robust to noisy input. We trained on the actual fMRI story text rather than an external corpus, so the resulting embeddings capture patterns specific to these stories.

3.2.3 Hyperparameter tuning

This stage involves two models needing tuning (the encoder and the downstream ridge regression) with a combinatorial hyperparameter space if tuned jointly. We instead tuned them sequentially: split the training stories into training/evaluation subsets, tune the encoder on that split, retrain the encoder on the full training set, then re-split the training set again for ridge’s own regularization tuning. This avoids the $x \times y$ blow-up of jointly tuning x encoder configurations against y ridge configurations, at the cost of not jointly optimizing both.

We searched hidden size $\{128, 256, 512\}$ (BERT-base uses 768), a fixed 4 attention heads (BERT-base uses 12; reduced for a smaller model given limited data), number of layers $\{2, \dots, 6\}$ (BERT-base uses 12), intermediate size fixed at $4 \times$ hidden size, and learning rate in $[1e-5, 5e-5]$, using Optuna, minimizing MLM evaluation loss. Table 2 shows the top two and worst configurations found.

Rank	Hidden	Heads	Layers	Learning rate	Epochs	Eval. loss
Top 1	512	4	4	1.3459e-05	2	6.8925
Top 2	512	4	2	2.4305e-05	2	6.9337
Worst	256	4	5	3.0972e-05	8	9.6497

Table 2: Hyperparameter search results for the custom encoder (validation loss after training).

The best configuration reaches its lowest evaluation loss after only two epochs, after which it overfits rapidly (Figure 1): an expected consequence of pretraining a transformer on a text corpus this small (a few thousand words per story, well short of the corpora typically used to pretrain BERT-scale models).

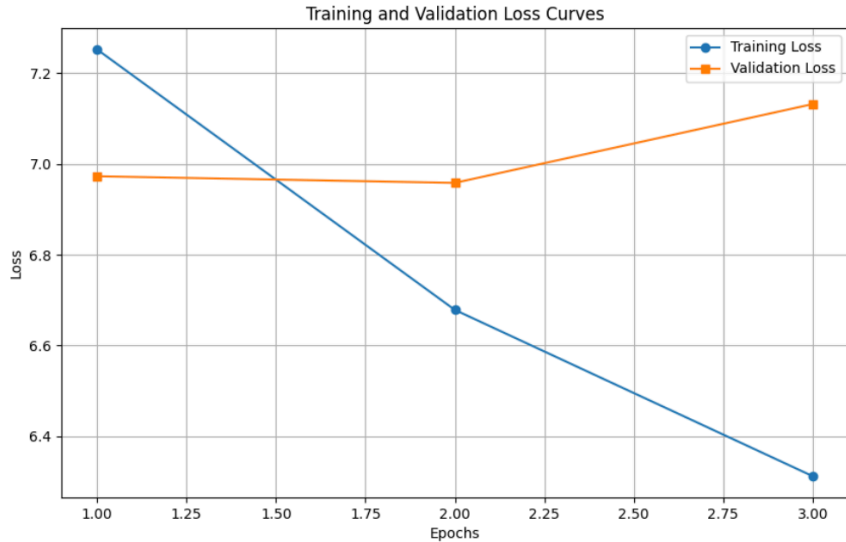


Figure 1: Training and validation loss curves for the custom encoder.

3.3 Large-scale pretrained and LoRA fine-tuned BERT

The custom encoder above is contextual but only ever sees a few thousand words of story text, far too little to learn general-purpose language structure. We next used **bert-base-uncased**, pretrained by Google on a much larger corpus via MLM and next-sentence prediction, to test whether contextual representations only pay off once they carry that outside knowledge.

The hidden size of **bert-base-uncased** is 768; with lags $[1, 2, 3, 4]$ this gives a 3072-dimensional feature per fMRI scan. Embeddings are extracted, downsampled, and lagged exactly as in Section 3.2. Because the resulting feature count can exceed the number of fMRI scans available per story ($n \ll p$), ridge regression is a natural fit.

3.3.1 Fine-tuning with LoRA

We hypothesized that fine-tuning **bert-base-uncased** specifically on the fMRI story text could improve on the off-the-shelf pretrained embeddings further. Fully fine-tuning all 110M parameters would be expensive and prone to overfitting on our limited data, so we used Low-Rank Adaptation (LoRA): the pretrained weights $W_0 \in \mathbb{R}^{d \times k}$ are frozen, and a trainable low-rank update is added,

$$W = W_0 + BA, \quad B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}, \quad r \ll \min(d, k),$$

so only a small fraction of parameters need to be trained while still adapting the model to our task.

We searched rank $r \in \{4, 8\}$, scaling factor $\alpha \in \{16, 32\}$, dropout $\in \{0.05, 0.1\}$, target modules $\in \{[\text{query}, \text{value}], [\text{query}, \text{key}, \text{value}]\}$, and learning rate $\in \{2\text{e-}4, 5\text{e-}4\}$ (batch size fixed at 4), evaluating five configurations by validation MLM loss after 3 epochs (Table 3).

Table 3: LoRA hyperparameter search results (average validation loss after 3 epochs).

Trial	Rank	Alpha	Dropout	Target modules	Learning rate	Val. loss
1	4	16	0.1	query, value	$2 \cdot 10^{-4}$	2.4762
2	8	32	0.1	query, value	$2 \cdot 10^{-4}$	2.4527
3	8	32	0.1	query, key, value	$2 \cdot 10^{-4}$	2.4471
4	8	32	0.1	query, key, value	$5 \cdot 10^{-4}$	2.4070
5	8	32	0.05	query, key, value	$2 \cdot 10^{-4}$	2.4597

Trial 4 (rank 8, $\alpha = 32$, dropout 0.1, all of query/key/value adapted, learning rate 5e-4) was best. Increasing rank from 4 to 8 helped (more capacity for task-specific adaptation); adapting the key projection alongside query/value helped further; raising the learning rate from 2e-4 to 5e-4 gave the single largest improvement, suggesting the pretrained model needed a fairly large nudge to adapt to fMRI-story text; and reducing dropout from 0.1 to 0.05 hurt performance, indicating some regularization is needed against overfitting on our small fine-tuning set. Using these settings, we trained the final LoRA adapter for 15 epochs on the full training set (final training loss 2.4219).

4 Modeling

All six embedding methods above are evaluated the same way: with ridge regression, predicting each voxel’s BOLD signal independently.

4.1 Ridge regression

For the standard linear model $Y = X\beta + \epsilon$, ridge regression solves

$$\hat{\beta}^{\text{ridge}} = \arg \min_{\beta} \{ \|y - X\beta\|_2^2 + \lambda \|\beta\|_2^2 \}, \quad \hat{\beta}^{\text{ridge}} = (X^T X + \lambda I)^{-1} X^T y, \quad (3)$$

adding a regularization strength λ relative to ordinary least squares, well suited to the $p \gg n$, multicollinear feature matrices produced by these embeddings. This requires a linear predictor-response relationship, independent observations, and constant error variance; we did not separately validate these assumptions.

4.2 Fitting procedure

We train one ridge model per voxel per embedding method per subject (94,251 voxels for subject 2; 95,556 for subject 3), using a 70:30 train:test split by story. For each voxel, we grid-search $\lambda \in \{10, 100, 1000\}$ with 3-fold cross-validation. Bag-of-Words required additional dimensionality reduction (Section 3.1.1) purely due to runtime constraints; the full grid search for all methods and both subjects was kept to roughly an hour of compute per run.

5 Results

Table 4 compares all six embedding methods by mean, top-5%, and top-1% voxel-wise correlation coefficient (CC) between predicted and actual BOLD signal, on held-out test stories.

	Bag-of-Words	Word2Vec	GloVe	Custom BERT	Pretrained BERT	LoRA-tuned BERT
Mean CC, Subject 2	0.0091	0.0101	0.0097	0.0092	0.0174	0.0167
Mean CC, Subject 3	0.0138	0.0162	0.0147	0.0163	0.0206	0.0232
Top 5% CC, Subject 2	0.0355	0.0356	0.0317	0.0293	0.0471	0.0461
Top 5% CC, Subject 3	0.0490	0.0498	0.0431	0.0460	0.0637	0.0619
Top 1% CC, Subject 2	0.0565	0.0525	0.0457	0.0426	0.0682	0.0671
Top 1% CC, Subject 3	0.0782	0.0747	0.0640	0.0669	0.0910	0.0890

Table 4: Correlation coefficient (CC) between predicted and actual BOLD signal, per subject and embedding method.

Two patterns stand out. First, none of the three static embeddings, nor the custom-trained encoder, clearly outperforms the others: mean CCs sit in a narrow band around 0.009–0.016, and only about 0.1% of voxels reach a correlation coefficient over 0.1 (a “weak” correlation on the PCS scale) for the best of these four methods. Second, switching to the large-scale pretrained BERT model produces a clear jump (roughly 60–90% higher mean CC than any static/custom-trained method on both subjects) while LoRA fine-tuning on top of that pretrained model gives, at best, a small further improvement (and is not consistently better than the plain pretrained model across both subjects). Figure 4 shows per-voxel correlation for the best-performing static embedding (Word2Vec); Figures 7 and 10 show the corresponding distributions for pretrained and LoRA fine-tuned BERT.

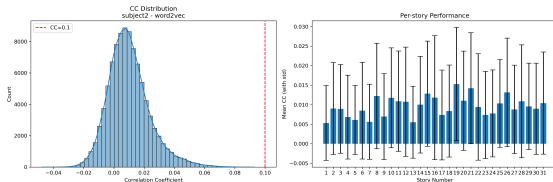


Figure 2: Subject 2

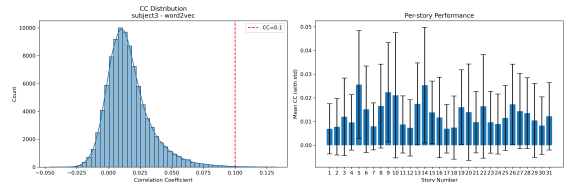


Figure 3: Subject 3

Figure 4: Per-voxel correlation coefficient, Word2Vec embedding (best-performing static embedding).

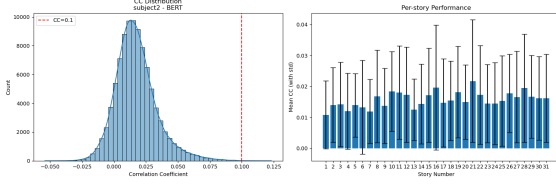


Figure 5: Subject 2

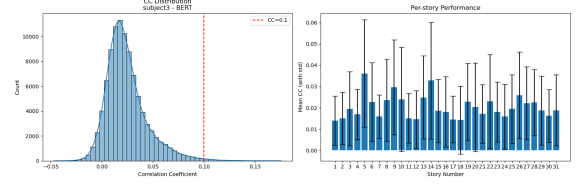


Figure 6: Subject 3

Figure 7: Ridge model evaluation, pretrained BERT embedding.

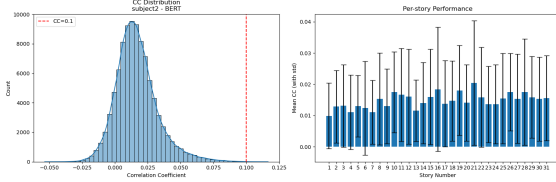


Figure 8: Subject 2

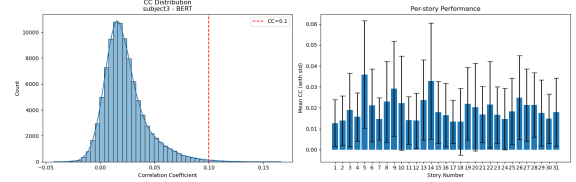


Figure 9: Subject 3

Figure 10: Ridge model evaluation, LoRA fine-tuned BERT embedding.

5.1 Stability across stories

To check whether these conclusions depend on which stories happened to land in the test split, we broke test-set performance down by individual story for the best static embedding (Word2Vec), computing the mean CC and a $[0.025, 0.975]$ quantile error band per story (Figure 13). For nearly every story, the error band crosses $CC=0$, meaning the null hypothesis “ $CC=0$ ” cannot be rejected story-by-story either, consistent with the pooled result that static and custom-trained embeddings carry very little predictive signal, and suggesting this conclusion is not an artifact of the particular train/test split used.

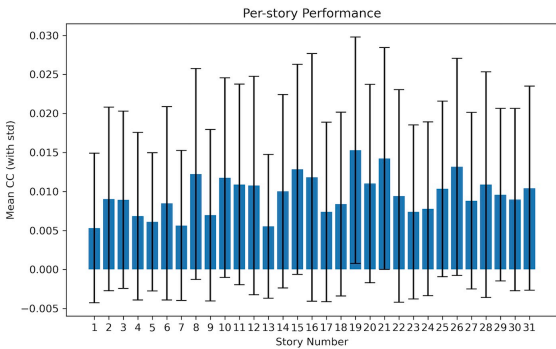


Figure 11: Subject 2

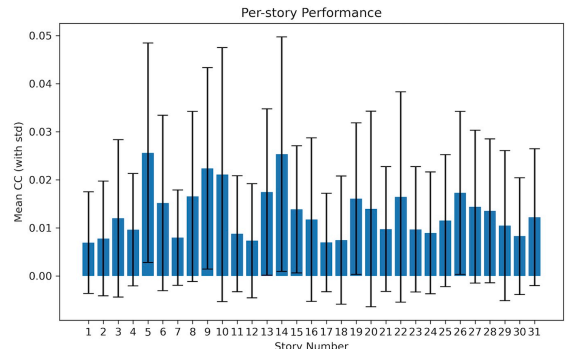


Figure 12: Subject 3

Figure 13: Per-story mean CC (Word2Vec embedding) with $[0.025, 0.975]$ quantile error bands.

By contrast, the pretrained and LoRA fine-tuned BERT models show error bands consistently

above zero for many stories: evidence that, unlike the static/custom-trained embeddings, these models capture a real, story-independent signal for at least some voxels.

6 Interpretation: SHAP and LIME

Given that the pretrained/fine-tuned BERT models are the only ones with a real predictive signal, we used SHAP and LIME to ask which words drive the best-predicted voxels of the LoRA fine-tuned model. We selected two test stories, identified their best-predicted voxels by ridge performance, and restricted interpretation to those voxels, in line with the “Predictivity” (P) principle of the PCS framework, so that attributions reflect real stimulus-response relationships rather than noise from unreliable predictions. Because the BERT model has a maximum input length, long passages were processed in overlapping chunks and attributions aggregated across them.

SHAP (SHapley Additive exPlanations) is a game-theoretic method that decomposes a model’s output for one instance into a baseline value plus feature-specific contributions that sum exactly to the prediction. Treating each word as a potential contributor to a voxel’s predicted activation, SHAP assigns each word a signed importance score.

LIME (Local Interpretable Model-agnostic Explanations) instead perturbs the input locally around one instance, queries the model, and fits a sparse local linear surrogate to approximate the model’s behavior near that instance. LIME trades SHAP’s consistency guarantees for speed, and, being purely local, sometimes disagrees with SHAP about which words matter most.

6.1 Test story 1: “Tetris”

For Subject 3 on the Tetris story, we examined the two best-predicted voxels, 127 and 128. For voxel 127, SHAP’s top 30 words all suppress predicted activation, with “become”, “too”, “powerful”, and “they” the strongest suppressors; among the largest individual contributions, video-game-related words (“gameboy”, “entertainment”, “system”) increase activation while difficulty/effort words (“do”, “can’t”, “attempt”, “overwhelms”) decrease it, consistent with a region more responsive to concrete, visual/semantic content than to negative or effortful concepts. LIME on the same voxel highlights “tetris” itself as the single largest positive contributor, along with “tagline”, “sequel”, and “smartphones”, while “people” and “that” skew negative.

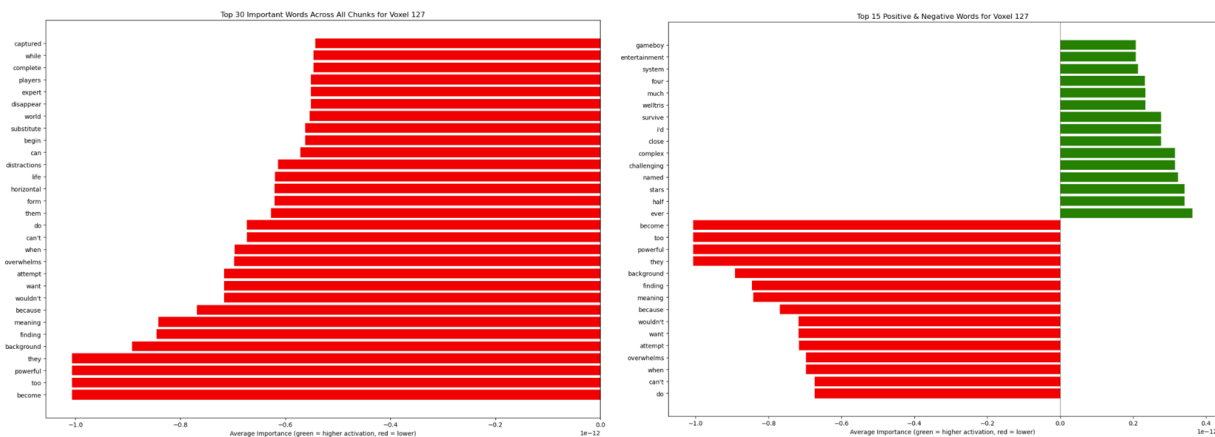


Figure 14: SHAP attributions, Test Story 1 (Tetris), Voxel 127.

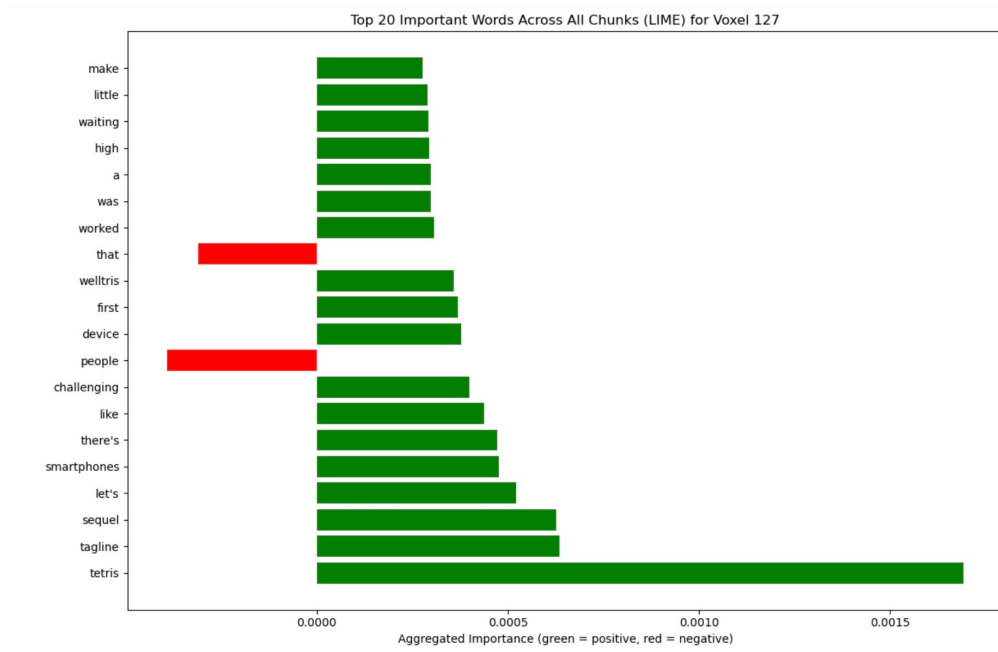


Figure 15: LIME attributions, Test Story 1 (Tetris), Voxel 127.

For voxel 128, SHAP highlights agency/role/goal-oriented words (“programmer”, “worker”, “let’s”, “start”, “player”) as positive drivers and complexity/conflict words (“mess”, “courts”, “involved”, “rich”) as negative drivers, suggesting a region more responsive to goal-directed action than to institutional or social complexity. LIME agrees only loosely, surfacing “i”, “arrival”, “the”, and “stars” as positive and “confronted”/“wouldn’t” as negative, including function words (“the”) that are hard to interpret as semantically meaningful.

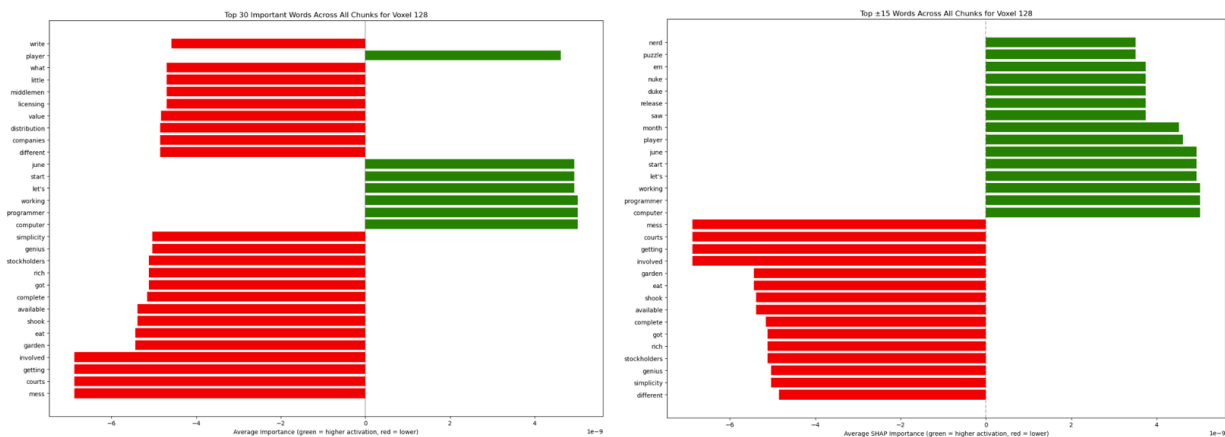


Figure 16: SHAP attributions, Test Story 1 (Tetris), Voxel 128.

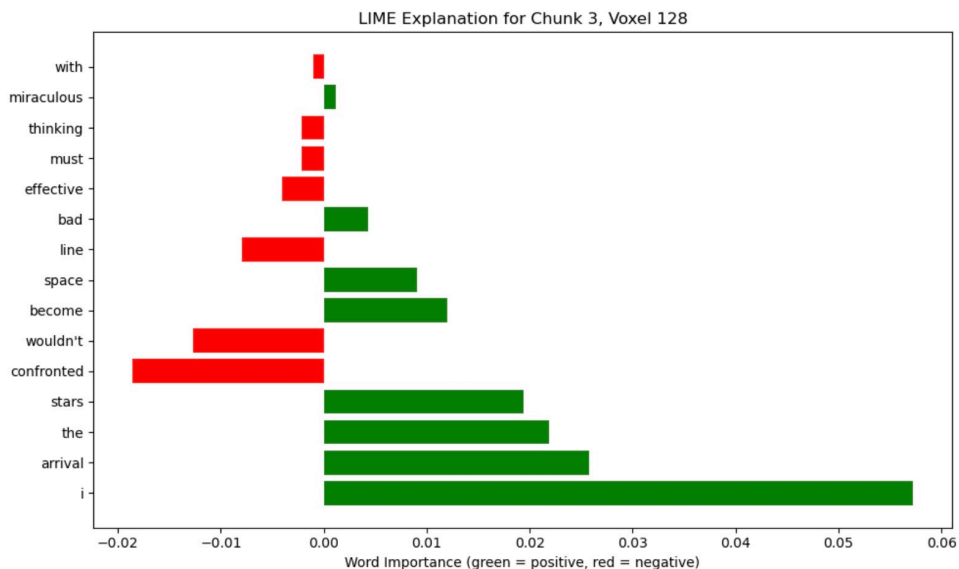


Figure 17: LIME attributions, Test Story 1 (Tetris), Voxel 128.

6.2 Test story 2: “My Father’s Hands”

For Subject 3’s best-predicted voxel (143) on this story, every word in the story carries a negative SHAP value, with “whenever”, “anywhere”, “car”, “held”, and “touched” the strongest suppressors: words tied to movement, physical interaction, and emotional moments. Since it is implausible for every word in a story spanning exposition, rising tension, and resolution to uniformly suppress a voxel’s response, this pattern more likely reflects subject- or voxel-specific baseline bias than a genuine word-level effect. LIME shows a similar dominance of negative attributions (“and”, “it”, “know”, “father’s”), with individual word effects that are difficult to interpret semantically.

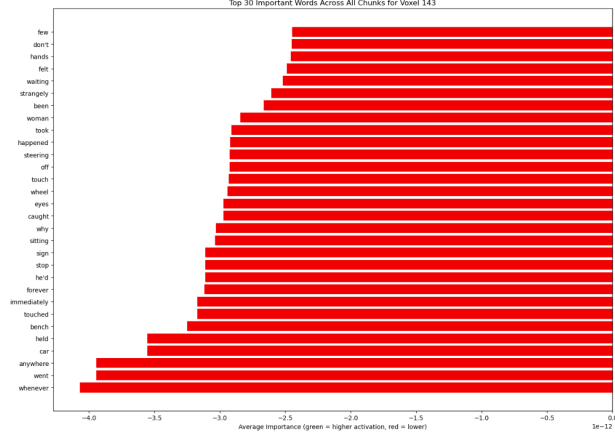


Figure 18: SHAP attributions, Test Story 2 (“My Father’s Hands”), Voxel 143.

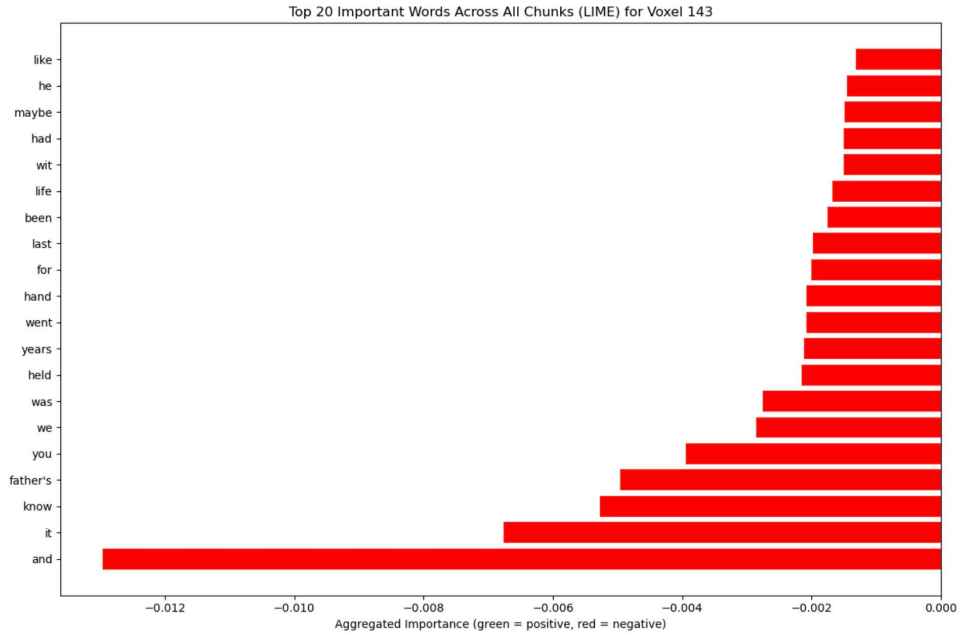


Figure 19: LIME attributions, Test Story 2 (“My Father’s Hands”), Voxel 143.

6.3 SHAP vs. LIME

Across both stories, SHAP and LIME frequently disagree about which words matter most for a given voxel, and it is often not obvious why a particular word receives a large attribution in either method. This means we cannot confirm the intuitive expectation that some small set of “obviously important” words drives each voxel’s response consistently across methods. Given that even the best-predicted voxels only achieve modest correlation coefficients, some of this disagreement likely reflects genuine noise in the underlying predictions rather than a real discrepancy between the two interpretation methods.

7 Conclusion

Across three stages of increasing embedding sophistication, a clear pattern emerges. Bag-of-Words, Word2Vec, and GloVe (all one-word-one-vector, context-free representations) produce almost no usable signal: mean correlation coefficients cluster near 0.01, and per-story error bands cannot reject $CC=0$ for almost any story. Making the embedding contextual without also making it large-scale did not help either: a custom BERT-style encoder trained from scratch via MLM on only the project’s story text performed no better than the static embeddings, most likely because a training corpus of a few thousand words per story is far too small to learn general contextual structure; its own validation loss begins increasing after just two epochs, a clear overfitting signature.

What did produce a real, story-independent signal was combining contextual architecture with large-scale pretraining: switching to **bert-base-uncased** raised mean CC by roughly 60–90% over every context-free or small-data method, and its per-story error bands are consistently above zero for many stories rather than centered on zero. LoRA fine-tuning that pretrained model on the story text gave, at best, a small further improvement, and was not consistently better than the plain pretrained model across both subjects, suggesting the corpus available for fine-tuning, while enough to nudge the model, is still far smaller than what pretraining supplied. SHAP/LIME analysis of the best-predicted voxels surfaced some plausible, thematically coherent word attributions (e.g. video-game vocabulary driving a Tetris-story voxel), but also attributions that are hard to interpret, disagree between methods, or are implausibly uniform across an entire story: a reminder that even the best models here explain only a small fraction of BOLD variance, and that interpretability results should be read with that ceiling in mind.

Overall, predicting brain activity from language remains difficult, and the main lever that mattered in this project was the scale of the corpus behind an embedding’s contextual knowledge, not context-awareness on its own or fine-tuning to the specific task.

8 Bibliography

- Jain, Shailee, and Alexander Huth. “Incorporating Context into Language Encoding Models for fMRI.” NeurIPS 2018.
- Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding” (arXiv:1810.04805)
- Sun et al., “How to Fine-Tune BERT for Text Classification?” (arXiv:1905.05583)
- Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models” (arXiv:2106.09685)
- Jurafsky and Martin, *Speech and Language Processing*, Ch. 11 (Masked Language Models)
- Hugging Face: Masked Language Modeling documentation
- Unsloth: LoRA Hyperparameters Guide
- Stack Exchange: good parameter ranges for BERT hyperparameters
- Wikipedia: Word2vec
- Wikipedia: GloVe
- Papers with Code: Skip-gram Word2Vec

- Suri, “A Dummy’s Guide to Word2Vec”
- “A Mathematical Introduction to GloVe Word Embedding”
- ScienceDirect: related work on word embeddings and individual differences